

Centro Universitário Senac-RS

ADS - Análise e Desenvolvimento de Sistemas / SPI - Sistemas para Internet

Unidade Curricular: Qualidade de Software

Prof.: Luciano Zanuz

Atividade PBL – Aula 12

BDD e Automação Orientada a Comportamento – LocalEats

Contexto

Após evoluir a qualidade do sistema LocalEats com:

- Planejamento de testes
- Testes manuais
- Testes unitários com TDD
- Testes funcionais automatizados

A equipe de Qualidade agora precisa avançar para um novo nível:

👉 Transformar requisitos e comportamentos do sistema em cenários executáveis utilizando BDD (Behavior-Driven Development).

Até agora:

- Os testes estavam muito focados na implementação
- Os fluxos eram descritos apenas tecnicamente
- Existia dificuldade em alinhar negócio, desenvolvimento e qualidade

Agora, o desafio é:

👉 Descrever o comportamento esperado do sistema de forma legível e automatizável.

O sistema ainda apresenta problemas como:

- Ambiguidade nos requisitos
- Dificuldade em validar comportamento esperado
- Falta de documentação viva
- Testes difíceis de entender por pessoas não técnicas
- Fragilidade em automações após mudanças na interface

A equipe precisa garantir que:

“Os comportamentos esperados do sistema estejam claramente documentados e automaticamente validados.”

Para isso, será adotado:

- BDD (Behavior-Driven Development)
- Escrita de cenários em Gherkin
- Automação orientada a comportamento
- Integração entre pytest-bdd e Playwright

👉 Link para o sistema LocalEats: <https://local-eats-unisenac.vercel.app/>

Objetivo da Atividade

Aplicar, de forma prática e orientada ao uso real:

- Escrita de cenários BDD
- Linguagem Gherkin
- Estrutura Given-When-Then
- Automação orientada a comportamento
- Uso de pytest-bdd
- Integração com Playwright
- Separação entre comportamento e implementação
- Organização e manutenção de testes automatizados

⚠️ **Importante**

O foco desta atividade NÃO é apenas automatizar cliques.

👉 O principal objetivo é:

Transformar regras e comportamentos do sistema em cenários compreensíveis por pessoas técnicas e não técnicas.

Utilizar obrigatoriamente:

- Python
- pytest
- pytest-bdd
- Playwright
- VSCode

👉 O mais importante é:

Compreender como o BDD aproxima:

- negócio

- qualidade
- desenvolvimento



Tarefas

Elaborem um documento contendo:

◆ 1. Fluxo escolhido

Selecionar um fluxo do sistema por integrante do grupo.



Cada integrante deverá trabalhar com um comportamento diferente.



Utilizar preferencialmente um dos fluxos abaixo:



1. Busca de restaurantes

O que faz

Permite pesquisar restaurantes ou culinárias.



Problema que resolve

Facilita encontrar opções rapidamente.



Importância

Fluxo principal de descoberta do sistema.



Cenários esperados

- Busca válida retorna resultados
- Busca inexistente retorna vazio
- Campo vazio mantém listagem



2. Navegação entre páginas

O que faz

Permite navegar entre:

- Explorar
- Favoritos
- Meus Pedidos



Importância

Garante navegação funcional do sistema.



Cenários esperados

- Página carrega corretamente
- Navegação ocorre sem erro
- Elementos principais aparecem



3. Histórico de pedidos

O que faz

Exibe pedidos realizados pelo usuário.



Importância

Fluxo importante de acompanhamento.



Cenários esperados

- Pedidos exibidos corretamente
- Status visível
- Valor total apresentado



4. Visualização de restaurantes

O que faz

Permite visualizar restaurantes disponíveis.



Importância

Base da experiência do usuário.



Cenários esperados

- Cards carregados
- Nome visível
- Imagem carregada



5. Filtro por categoria

O que faz

Permite filtrar restaurantes por culinária.



Importância

Melhora usabilidade e descoberta.



Cenários esperados

- Filtro aplicado corretamente

- Lista atualizada
- Categoria destacada

◆ 2. Escrita dos cenários BDD

Criar cenários utilizando Gherkin.

👉 Estrutura obrigatória:

```
Feature:  
Scenario:  
Given  
When  
Then
```

👉 Cada integrante deverá criar:

- mínimo de 2 cenários
- relacionados ao fluxo escolhido

💻 Exemplo

Feature: Histórico de pedidos

```
Scenario: Visualizar pedidos realizados  
  Given que o usuário acessa a página de pedidos  
  When visualizar o histórico de transações  
  Then o sistema deve exibir os pedidos cadastrados
```

◆ 3. Implementação da automação com pytest-bdd

Automatizar pelo menos 1 cenário.

👉 Estrutura mínima:

```
features/  
  historico_pedidos.feature
```

```
tests/  
  test_historico_pedidos.py
```

👉 O teste deve conter:

- Integração com pytest-bdd
- Steps Given/When/Then
- Assertions relevantes

- Navegação real no sistema

Exemplo (simplificado)

```
from pytest_bdd import scenarios, given, when, then

scenarios('../features/historico_pedidos.feature')

@given('que o usuário acessa a página de pedidos')
def acessar_pagina(page):
    page.goto('https://local-eats-unisenac.vercel.app/static/orders.html')

@when('visualizar o histórico de transações')
def visualizar(page):
    page.locator('text=Histórico de Transações').is_visible()

@then('o sistema deve exibir os pedidos cadastrados')
def validar(page):
    assert page.locator('text=Pedido #1').is_visible()
```

◆ 4. Organização do projeto

Organizar a automação utilizando boas práticas.

👉 Estrutura esperada:

```
projeto/
|
|— features/
|— tests/
|— evidencias/
```

◆ 5. Execução dos testes

Executar:

```
pytest
```

👉 Informar:

- Total de cenários
- Quantos passaram
- Quantos falharam

👉 Registrar:

- Evidência da execução
- Prints ou logs

◆ 6. Análise crítica

Responder:

- O cenário escrito ficou compreensível?
- O teste automatizado ficou legível?
- O BDD ajudou a entender o comportamento?
- Quais dificuldades surgiram?
- Os seletores foram frágeis?
- O teste ficou dependente da interface?
- O cenário representa realmente uma regra de negócio?
- O que tornaria o teste mais robusto?

◆ 7. Reflexão no contexto do LocalEats

Responder:

- BDD melhora comunicação entre equipe?
- Todo teste deve ser escrito em BDD?
- Quando vale a pena usar BDD?
- O comportamento ficou mais claro?
- Como isso ajuda no projeto do grupo?

Formato: arquivo Markdown (.md)

Entrega: repositório do grupo no GitHub

/aula-12-bdd-automacao-comportamento.md

👉 Trabalho individual ou em grupo (até 4 integrantes)



Exemplo

<https://github.com/lucianozanuz/pbl-qualidade-software-2026-1/blob/main/pbl/aula-12-bdd-automacao-comportamento.md>



Avaliação (Rubrica – Unisenac-RS)



D — Não atingiu

- Cenários incompletos
- Estrutura Gherkin incorreta
- Automação não executa

C — Parcial

- Cenários pouco claros
- Automação parcial
- Pouca organização

B — Pleno

- Cenários corretos
- Automação funcional
- Boa separação Given-When-Then

A — Excelência

- Cenários claros e legíveis
- Boa representação de regras de negócio
- Código organizado
- Assertions relevantes
- Boa integração pytest-bdd + Playwright
- Automação robusta
- Excelente análise crítica



Dica final

Para obter conceito A, vocês devem:

- Escrever cenários pensando no negócio
- Evitar detalhes excessivamente técnicos
- Criar automações legíveis
- Usar bons seletores
- Organizar corretamente o projeto
- Pensar em manutenção futura



Mentalidade esperada:

“Uma pessoa não técnica conseguiria entender o comportamento descrito?”